DEFCON 2.0: Attack-Defense Container Development Guidebook

This guidebook dictates the architecture, vulnerability design, and thematic deployment profiles for the DEFCON 2.0 single-container environments. Because participants will utilize high-capability AI (GPT-6 Astra), standard CTF environments will be exploited and patched automatically within minutes. The objective is to build robust, self-contained architectures that force human intuition, complex reasoning, multi-step execution, and state manipulation.

Section 1: Core Architecture & Infrastructure Requirements

Since all components—terminal access, offensive tools, vulnerable services, and administrative daemons—must exist within a single Docker container per team, the environment must be orchestrated like a lightweight virtual machine.

1. Process Management & Access

- **The Supervisor:** You must use a process supervisor (e.g., supervisord or s6-overlay) as the container's ENTRYPOINT. It will simultaneously run ttyd, the web services, databases, and the flag rotation daemon.
- **Web Terminal (ttyd):** Bind ttyd to a dedicated port (e.g., 8080). Authentication is done outside the container by the admin, ttyd just serves the terminal. However, the workspaces of different team mates should not interfere with each other. It must spawn a shell as a non-root user (e.g., ctf-player).
- **Privilege Segregation:** Do not run web services as root. Each application must run under its own user account (e.g., svc-webapp). If an attacker achieves Remote Code Execution (RCE), they only compromise that specific service user, forcing lateral privilege escalation.

2. Administrative Daemons

- **The Flag Rotator:** A background script (running as root or a dedicated flag-admin user) that executes every 3-5 minutes. It generates a new flag, injects it into specific files or databases owned by the vulnerable services, and securely pushes the flag hash to your central scoring API.
- **The Rollback Mechanism (Self-Healing):** Teams will inevitably break their services while patching.
  - Store a pristine, read-only copy of all service source code in /opt/backups/.
  - Provide a utility script (/usr/bin/restore-service <service_name>) usable by the ctf-player user.
  - The script must use restricted sudo rules (no password required for this specific command) to stop the broken service via supervisorctl, wipe the active directory, copy the pristine files back from /opt/backups/, and restart the service.

3. The Developer's Pre-Flight Checklist

Before a container is finalized, it must pass these checks:

1. Can supervisord keep all services and the terminal running concurrently?
2. Can the service be patched using only terminal tools without breaking core functionality?
3. Does the rollback script successfully restore a deleted service?
4. **AI Stress Test:** If an LLM is given the source code and provides a working exploit on the first try without needing human guidance, the vulnerability is too simple and must be obfuscated.

Section 2: The Vulnerability Catalog (AI-Resistant Design)

LLMs excel at static code analysis and single-shot exploits. To defeat AI, you must build vulnerabilities requiring state manipulation, chained exploits, and intentional misdirection ("Red Herrings").

Tier 1: Easy (Requires human intuition to spot)

1. **Insecure Direct Object Reference (IDOR):** An endpoint /api/profile?user_id=101 returns user data. The flag is stored in the profile of user_id=1.
2. **Filtered Local File Inclusion (LFI):** A script uses include($_GET['page'] . '.php'). Bypass fake WAF rules using null byte injection or path traversal (../../../../../etc/flag.txt%00).
3. **Base64 Cookie Manipulation:** The application assigns a cookie like session=dXNlcg== (base64 for "user"). Encode "admin" to elevate privileges.
4. **Misconfigured CORS:** An API endpoint leaks sensitive information to any origin (Access-Control-Allow-Origin: *), requiring the attacker to host a malicious HTML page.
5. **Git History Disclosure:** The web root accidentally exposes the .git/ directory. Teams must use git log to find hardcoded credentials removed in previous commits.
6. **Directory Traversal (Static Server):** A flawed path resolver in an Express or Python static server allows users to escape the web root via ?file=../../../flag.txt.
7. **Command Injection in Diagnostic Tools:** A "Ping IP" utility passes input directly to a shell (exec("ping -c 4 " + ip)), allowing command concatenation.
8. **Weak Session IDs:** The application generates session IDs using predictable hashes, such as md5(username).
9. **Backup File Leftover:** The developer left config.php.bak in the web root, exposing database credentials.
10. **Reflected XSS into Admin Bot:** A search page reflects input. Teams must craft an XSS payload and submit the URL to a simulated headless "Admin Bot" to steal its session cookie.

Tier 2: Medium (Requires chaining distinct flaws)

1. **Blind SQLi + Strict Filter Evasion:** A login form is vulnerable to SQL injection, but a WAF blocks spaces, SELECT, and UNION. Teams must construct boolean-based blind payloads using alternative characters (/**/, ||).
2. **SSTI with Sandboxing:** A template engine (like Jinja2) evaluates user input, but common exploit keywords (__class__) are filtered, requiring hex-encoded bypasses or traversing application globals.

3. **SSRF to Internal Services:** A "URL Preview" feature allows Server-Side Request Forgery. Teams must bypass IP filters (e.g., using decimal IPs) to interact with an unauthenticated internal Redis instance bound to 127.0.0.1 and write a web shell.
4. **Type Juggling:** An API uses loose comparison (==) for checking a JSON-provided password hash. Providing a boolean true or an integer 0 bypasses the check.
5. **Out-Of-Band (OOB) XXE:** An endpoint parses XML blindly but returns no output. Teams must host an external DTD to force the server to exfiltrate local files via outbound HTTP requests.
6. **Insecure Deserialization:** The application unserializes data from a cookie. Teams must analyze the source code to find a hidden "gadget" class to trigger execution.
7. **Second-Order SQL Injection:** Users safely update their profile (parameterized INSERT), but a backend reporting script blindly concatenates that saved profile data into a new SELECT query.

Tier 3: Hard (Business Logic & Cryptographic State)

1. **Race Condition (TOCTOU):** A marketplace feature checks a user's balance, lags for a split second, and then deducts funds. Sending 50 simultaneous purchase requests causes the database to read the balance 50 times before the first deduction, resulting in infinite generated assets.
2. **Custom Crypto Padding Oracle:** The application uses a flawed custom implementation of AES-CBC. Teams must script an attack that analyzes HTTP 500 "Padding" vs "Decryption" errors to forge an admin token.
3. **Prototype Pollution to RCE:** A Node.js application recursively merges user-provided JSON. Sending __proto__ overwrites environment variables (like NODE_OPTIONS) used when spawning legitimate child processes.
4. **Chained DOM XSS to Internal WebSocket:** A frontend dashboard has a DOM XSS flaw. Teams must use it to send a malicious binary payload (via JavaScript) to an internal Java WebSocket server listening on localhost, triggering deserialization RCE.

Section 3: The 4 Container Profiles (Deployment Boxes)

To maximize replayability, package the vulnerabilities into four distinct Container Profiles. Each team can be assigned a different box, or the infrastructure can rotate them at the 2-hour mark. Each box utilizes "Red Herrings"—fake vulnerabilities designed to waste AI compute.

Box 1: "The Skeletonized Movement"

Theme: An underground marketplace for custom, kinetic art and skeletonized automatic watch parts. The vibe is industrial, precise, and clockwork.**Architecture:** Python Flask frontend, SQLite database, internal Redis.**Red Herrings:** A visually complex React frontend with heavy client-side state obfuscation; fake JWT tokens stored in localStorage that are just random bytes.

- **Easy (5):** IDOR on Order History, Filtered LFI (blueprint viewer), Base64 Cookie Manipulation, Git History Disclosure, Misconfigured CORS on kinetic design specs.
- **Medium (4):** Blind SQLi + Strict Filter (part search), SSTI in Engraving Preview, SSRF to

Internal Redis (manufacturer URL fetcher), Type Juggling (VIP access PIN).
- **Hard (2):** Race Condition (TOCTOU on buying "Limited Edition Escapement" parts), Custom Crypto Padding Oracle (VIP session tokens).

Box 2: "The Fuzz Pedal"

Theme: A gritty gear-exchange and forum for analog guitar modifications, vintage amplifiers, and custom sound engineering.**Architecture:** Legacy PHP-FPM, internal headless Chrome (Admin Bot), MySQL.**Red Herrings:** An incredibly secure bcrypt login implementation; a fake WAF rule blocking SELECT on parameters not connected to a database.

- **Easy (5):** Command Injection ("Ping Amp IP"), Weak Session IDs (forum login), Backup File Leftover (schematics database), Reflected XSS (guitar tab search), Directory Traversal (static amp schematics).
- **Medium (4):** Second-Order SQLi (band bio updates), OOB XXE (invoice generator), PHP Object Injection (saved pedalboard cookies), SSRF to Localhost (album art fetcher).
- **Hard (2):** Chained DOM XSS to Internal WebSocket (HTML5 MIDI dashboard to Java backend), JWT Algorithm Confusion (LFI to steal public key).

Box 3: "The Bootlegger's ROM"

Theme: An underground hub for customizing mobile OS interfaces, swapping out Android launchers to look like iOS, and flashing unauthorized UI skins.**Architecture:** Node.js/Express, Python background workers, internal API endpoints.**Red Herrings:** An endpoint /api/exec?cmd= protected by a mathematically impossible visual CAPTCHA; a fake internal port (tarpit) that drops connections.

- **Easy (5):** Information Disclosure (.env.old), IDOR on Premium Skins, Reflected XSS (device model parameter), Base64 Cookie Manipulation (beta tester role), Filtered LFI (icon pack renderer requiring double-url-encoding).
- **Medium (4):** Blind SQLi + Strict Filter (widget search), SSTI with Sandboxing (custom CSS template engine), Type Juggling (theme activation keys), OOB XXE (vector icon uploads).
- **Hard (2):** Prototype Pollution to RCE (JSON skin configuration), Race Condition in File Upload (shell uploaded and executed in the 1-second scanning window).

Box 4: "The Neon Dhaba"

Theme: A cyberpunk-themed late-night food delivery service specializing in synthesized North Indian street food and spice-trade logistics.**Architecture:** Ruby on Rails backend, PostgreSQL, internal Memcached.**Red Herrings:** An unused, highly vulnerable cryptography library left in Gemfile.lock; complex, mathematically sound discount code generation logic.

- **Easy (5):** Backup File Leftover (database.yml.bak), Directory Traversal (menu image server), Weak Session IDs, Misconfigured CORS (delivery route API), IDOR on Order Receipts.
- **Medium (4):** Insecure Deserialization (saved cart state), SSRF to Internal Services (restaurant webhook tester), Blind SQLi + Strict Filter (spice inventory search), Second-Order SQLi (delivery instruction fields).
- **Hard (2):** Race Condition (TOCTOU on applying promotional credits to orders), Custom Crypto Padding Oracle (premium delivery tier tokens).